

Data Analysis

Data Analysis

Descriptive Statistics

Department of Economics

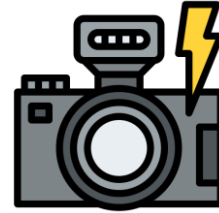
Ivan Bondarev



Email

- i.bondarev@inno.mgimo.ru

Signs



– Please take pictures of the information. Medium priority.

Source: https://www.flaticon.com/ru/free-icon/camera-flash_3460307



– Please write down the information. High priority.

Source: <https://openmoji.org/library/emoji-270D/>

TOPICS

- 1.Preparing the data
- 2.Describing one numerical variable
- 3.Describing categorical variables
- 4.Comparing groups
- 5.Two variables together

Useful sources

- Github - Nikita Artamonov

<https://github.com/artamonoff>



Useful sources

- Github – Ivan Bondarev

[https://github.com/BondarevIM/MGIMO-SGIA-Data-](https://github.com/BondarevIM/MGIMO-SGIA-Data-Analysis)

[Analysis](https://github.com/BondarevIM/MGIMO-SGIA-Data-Analysis)



Describing a dataset

Two diamond shops both report an average price of 5,000 SGD.

Are their price lists the same?

What would you need to know to answer?

What descriptive statistics do

- A dataset of 308 rows is too large to read row by row
- Descriptive statistics replace it with a few numbers
- Three questions are asked about every variable:
 - o Where is the centre?
 - o How wide is the spread?
 - o What is the shape of the distribution?
- Nothing here proves anything. It describes, it does not explain

The dataset: Diamond.csv

- 308 observations, 5 variables, no missing values
 - o carat – weight of the stone
 - o colour – D, E, F, G, H, I (D is the best)
 - o clarity – IF, VVS1, VVS2, VS1, VS2 (IF is the best)
 - o certification – GIA, HRD, IGI (the laboratory)
 - o price – price in Singapore dollars

Source: <https://jse.amstat.org/v9n2/datasets.chu.html>

Check the data before you count

```
df = pd.read_csv("Diamond.csv")
```

```
df.shape
```

```
df.dtypes
```

```
df.isnull().sum()
```

```
df.head(10)
```

A statistic computed on a table you have not looked at means nothing

Two kinds of columns

```
numerical_cols = ['carat', 'price']  
categorical_cols = ['colour', 'clarity',  
                    'certification']
```

- Numerical columns get a mean, a median, a standard deviation
- Categorical columns get counts and a mode
- The average of GIA, HRD and IGI does not exist

Sample size, minimum, maximum

```
len(df)
```

```
df[numerical_cols].min()
```

```
df[numerical_cols].max()
```

- carat runs from 0.18 to 1.10, price from 638 to 16,008
- The first job of min and max is to catch impossible values:
 - o a price of 0, a negative weight, a year of 20222

The mean

```
df[numerical_cols].mean()
```

- carat 0.63, price 5,019.48
- The mean is the balance point of the data
- Every observation pulls on it, so one extreme value moves it
- Add one stone at 100,000 SGD and the average price jumps

The median

```
df[numerical_cols].median()
```

- carat 0.62, price 4,215
- Sort the values and take the middle one
- Half the stones cost less than 4,215 SGD, half cost more
- The median does not care how far away the extremes are

Mean against median

price: mean 5,019, median 4,215

carat: mean 0.63, median 0.62

*When the mean is far above the median,
the distribution has a long tail to the right*

A few expensive stones pull the average price up; carat is almost symmetric

Variance and standard deviation

```
df[numerical_cols].var(ddof=1)
```

```
df[numerical_cols].std(ddof=1)
```

- Standard deviation of price: 3,403 SGD
- Variance is measured in squared units, so it is hard to read
- The standard deviation is its square root, in the original units
 - o ddof=1 divides by n-1: we have a sample, not the whole market

Quartiles and percentiles

```
df[numerical_cols].quantile([0.25, 0.50, 0.75])
```

- price: Q1 = 1,625, Q2 = 4,215, Q3 = 7,446
- 25 per cent of the stones cost less than 1,625 SGD
- Q2 is the median, by definition
 - o quantile([0.05, 0.95]) describes the tails: 919 and 10,656

The interquartile range

```
IQR = df['price'].quantile(0.75) - df['price'].quantile(0.25)
```

- IQR of price = 7,446 - 1,625 = 5,821 SGD
- It is the width of the middle half of the data
- A common rule marks a value as an outlier if it lies below
 - $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$

Upper bound 16,177, most expensive stone 16,008: no outliers here

The shape of the distribution

```
df[numerical_cols].skew()
```

```
df[numerical_cols].kurtosis()
```

- Skewness of price 0.66: the long tail points to the right
- Skewness of carat 0.01: almost symmetric
- Kurtosis says how heavy the tails are compared with a normal curve
 - o pandas reports excess kurtosis: 0 means normal, not 3

Source: pandas.pydata.org/docs/reference/api/pandas.DataFrame.kurtosis.html

All of it in one line

```
df[numerical_cols].describe().round(2)
```

- count, mean, std, min, 25%, 50%, 75%, max
- describe() gives in one line what we have just counted by hand
- Count first, understand, then use the shortcut
 - o round(2) keeps pandas from printing 1.158120e+07

Source: pandas.pydata.org/docs/reference/api/pandas.DataFrame.describe.html

Categorical variables

The colour column holds the letters D to I.

What is its average?

How would you describe such a column with numbers?

Frequency tables

```
df['certification'].value_counts().sort_index()
```

- GIA 151, HRD 79, IGI 78, total 308
- A frequency table is the whole description of a categorical column
- It shows the most common category and any empty ones
 - o `sort_index()` orders the categories; without it pandas orders by count

Shares instead of counts

```
df['certification'].value_counts(normalize=True)
```

- GIA 49.0%, HRD 25.6%, IGI 25.3%
- Almost half the stones are certified by GIA
- Shares are easier to compare than counts
- They also let you compare datasets of different size

The mode

```
df[categorical_cols].mode()
```

- colour F, clarity VS1, certification GIA
- The mode is the value that occurs most often
- For a categorical column it is the natural centre
- For a continuous column it is weak: repeated values are accidental
 - o A column can have several modes; mode() returns all of them

Statistics by group

```
df.groupby('certification')['price']  
    .agg(['mean', 'median', 'count'])
```

GIA 5,310 HRD 7,181 IGI 2,267

- One command answers: is price related to the laboratory?
- Split the rows into groups, compute the same statistic in each
- This is the most useful single command of the whole course

A group mean can mislead

- Mean price by colour, from the best colour to the worst:
D 7,100 E 5,508 F 4,786 G 4,426 H 4,965 I 5,175
- The worst colours are not the cheapest. Why?
- Because the groups differ in weight as well as in colour
- A group mean compares groups, not one feature at a time

Two variables together

Do heavier diamonds cost more?

*How would you show it with a single number,
without drawing anything?*

Covariance

```
df[numerical_cols].cov()
```

- Covariance of carat and price: 891.15
- Positive: heavy stones tend to be the expensive ones
- Negative would mean the opposite
- But what does 891.15 mean? Is that a lot?

Why covariance is not enough

```
df['price_k'] = df['price'] / 1000
```

- The same stones, the price written in thousands:
 - o covariance falls from 891.15 to 0.89
 - o correlation stays at 0.9447
- Covariance depends on the units, so it cannot be compared
- Correlation is covariance divided by both standard deviations

Correlation

```
df['carat'].corr(df['price'])
```

- 0.9447: a very strong positive linear relationship
- The value always lies between -1 and +1
- 0 means no linear relationship, not no relationship at all
- Correlation is not causation: weight does not have to be the cause

Two categorical variables

```
pd.crosstab(df['colour'], df['certification'])
```

- The table counts how often each pair of categories occurs
- GIA certifies stones of every colour; IGI is concentrated in a few
- If the columns had the same pattern, the two variables would be unrelated

Summary

- Check the table before computing anything
- Centre: mean is sensitive to extremes, median is not
- Spread: standard deviation, quartiles, interquartile range
- Shape: mean above median means a tail to the right
- Categorical columns: frequencies, shares, mode
- groupby compares groups; corr measures a linear link, not a cause
- describe() is the shortcut, not the starting point